

MI LDC API

Version 3.2

© 2021 SigmaStar Technology Corp. All rights reserved.

SigmaStar Technology makes no representations or warranties including, for example but not limited to, warranties of merchantability, fitness for a particular purpose, non-infringement of any intellectual property right or the accuracy or completeness of this document, and reserves the right to make changes without further notice to any products herein to improve reliability, function or design. No responsibility is assumed by SigmaStar Technology arising out of the application or use of any product or circuit described herein; neither does it convey any license under its patent rights, nor the rights of others.

SigmaStar is a trademark of SigmaStar Technology Corp. Other trademarks or names herein are only for identification purposes only and owned by their respective owners.

REVISION HISTORY

Revision No.	Description	Date
3.0	Initial release	12/04/2020
3.1	添加 MI_LDC_SetOutputPortAttr 接口	07/04/2021
	新增 PROCFS 介绍	08/25/2021
3.2	- 添加 Maruko 平台信息 - 添加新接口： - MI_LDC_AttachToChn - MI_LDC_DetachFromChn	04/01/2022

TABLE OF CONTENTS

REVISION HISTORY i

TABLE OF CONTENTS ii

1. 概述 1

1.1. 模块说明.....	1
1.2. 流程框图.....	1
1.3. 各平台特性.....	1
1.4. 关键字说明.....	2

2. API 参考 3

2.1. MI_LDC_CreateDevice.....	3
2.2. MI_LDC_DestroyDevice.....	5
2.3. MI_LDC_CreateChannel.....	6
2.4. MI_LDC_DestroyChannel.....	6
2.5. MI_LDC_StartChannel.....	7
2.6. MI_LDC_StopChannel.....	8
2.7. MI_LDC_GetOutputPortAttr.....	8
2.8. MI_LDC_SetChnParam.....	9
2.9. MI_LDC_DoLutDirectTask.....	10
2.10. MI_LDC_SetOutputPortAttr.....	12
2.11. MI_LDC_AttachToChn.....	13
2.12. MI_LDC_DetachFromChn.....	13

3. 数据类型 15

3.1. MI_LDC_DEV.....	15
3.2. MI_LDC_CHN.....	15
3.3. MI_LDC_DevAttr_t.....	16
3.4. MI_LDC_WorkMode_e.....	16
3.5. MI_LDC_MapInfoType_e.....	17
3.6. MI_LDC_DisplacementMapInfo_t.....	17
3.7. MI_LDC_SensorCalibInfo_t.....	18
3.8. MI_LDC_ChnAttr_t.....	18
3.9. MI_LDC_ChnParam_t.....	19
3.10. MI_LDC_OutputPortAttr_t.....	20
3.11. MI_LDC_DirectBuf_t.....	21
3.12. MI_LDC_LutTaskAttr_t.....	21

4. 错误码 23

5. PROCFS 介绍 24

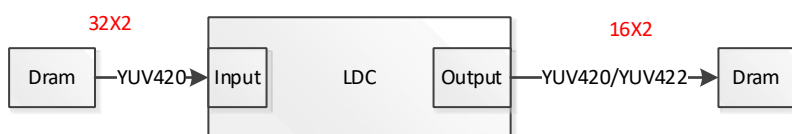
5.1. cat.....	24
5.2. echo.....	26

1. 概述

1.1. 模块说明

镜头畸变矫正(Lens distortion correction), 目前模块支持功能包括镜头畸变矫正、图像拼接与融合、电子防抖, 查找表功能。

1.2. 流程框图



注:

1、**32X2**, **16X2** 分别为输入图像, 输出图像宽高对齐限制。

1.3. 各平台特性

各平台支持的功能:

Device 0

平台	功能	镜头畸变矫正	图像拼接与融合	电子防抖	查找表
Tiramisu		Y	Y	Y	N
Muffin		Y	Y	Y	Y
Mochi		Y	Y	Y	Y
Maruko		Y (Horizontal)	N	N	N

Device 1

平台	功能	镜头畸变矫正	图像拼接与融合	电子防抖	查找表
Muffin		Y	Y	Y	Y

功能介绍:

镜头畸变矫正

根据镜头标定参数, 对输入图像进行畸变矫正处理, 然后按照用户配置的投影模式以及感兴趣区域进行输出。

图像拼接与融合

针对多 sensor 输入的图像进行畸变矫正处理, 然后将多个图像拼接在一起, 最后将图像重叠区域进行融合处理。

数字图像防抖:

目前支持陀螺仪算法防抖, 即根据陀螺仪产生的数据计算当前帧的图像的运动偏移情况, 然后根据运动偏移情况对当前图像进行平移、旋转等变换, 从而起到防抖的效果。

查找表

提供二维查找表功能，x,y 方向上索引和表的数据位宽都是 8bit,可以用于图像融合算法权重表的快速查找。

1.4. 关键字说明

- device
Ldc 对应的硬件设备。
- channel
Ldc 模块处理通道，各通道分时复用 LDC 硬件，目前仅支持单通道。

2. API 参考

表 2-1 功能模块 API 列表

API 名	功能
MI_LDC_CreateDevice	创建 LDC device 设备
MI_LDC_DestroyDevice	销毁 LDC device 设备
MI_LDC_CreateChannel	创建 LDC channel 通道
MI_LDC_DestroyChannel	销毁 LDC channel 通道
MI_LDC_StartChannel	启用 LDC channel 通道
MI_LDC_StopChannel	禁用 LDC channel 通道
MI_LDC_GetOutputPortAttr	获取 LDC output 属性
MI_LDC_SetChnParam	配置 LDC 通道属性
MI_LDC_DoLutDirectTask	执行直接的查表任务，将 buf 直接送给 ldc 硬件处理
MI_LDC_SetOutputPortAttr	配置 LDC output 属性
MI_LDC_AttachToChn	将 LDC 通道绑定到对应模块通道
MI_LDC_DetachFromChn	将 LDC 通道解绑定

2.1. MI_LDC_CreateDevice

➤ 描述

创建一个 LDC device 设备。

➤ 语法

```
MI_S32 MI_LDC_CreateDevice(MI_LDC_DEV devId, MI\_LDC\_DevAttr\_t *pstDevAttr);
```

➤ 参数

参数名称	描述	输入/输出
devId	Device id 号	输入
pstDevAttr	Device 属性	输入

➤ 返回值

返回值 $\left\{ \begin{array}{l} 0 \text{ 成功。} \\ \text{非 } 0 \text{ 失败，参照[错误码](#)。} \end{array} \right.$

➤ 需求

- 头文件：mi_ldc.h

- 库文件：libmi_ldc.a

※ 注意

无。

➤ 举例

LDC 初始化设置，以及退出范例：

Tiramisu、Muffin、Mochi 平台

```
MI_S32 s32Ret = MI_SUCCESS;
MI_LDC_DEV LdcDevId = 0;
MI_LDC_CHN LdcChnId = 0;
MI_LDC_DevAttr_t stDevAttr = {0};
MI_LDC_ChnAttr_t stChnAttr = {};
MI_LDC_SensorCalibInfo_t * pstCalibInfo = &stChnAttr.stCalibInfo;

stChnAttr.eMode = MI_LDC_WORKMODE_LDC ;
s32Ret = ReadBufFromFile("ldc.json", stChnAttr.pConfigAddr, stChnAttr.u32ConfigSize);
if (s32Ret)
{
    return s32Ret;
}

stChnAttr.eInfoType = MI_LDC_MAPINFOTYPE_SENSORCALIB;
s32Ret = ReadBufFromFile("CalibPoly.bin", pstCalibInfo.pCalibPolyBinAddr, pstCalibInfo.u32CalibPolyBinSize);
if (s32Ret)
{
    return s32Ret;
}

MI_LDC_CreateDevice(LdcDevId, &stDevAttr);
MI_LDC_CreateChannel(LdcDevId, LdcChnId, &stChnAttr);
MI_LDC_StartChannel(LdcDevId, LdcChnId);

ReleaseBuf(stChnAttr.pConfigAddr);
ReleaseBuf(pstCalibInfo.pCalibPolyBinAddr);

/*销毁通道*/
MI_LDC_StopChannel(LdcDevId, LdcChnId);
MI_LDC_DestroyChannel(LdcDevId, LdcChnId);
MI_LDC_DestroyDevice(LdcDevId);
```

Maruko 平台

```
MI_S32 s32Ret = MI_SUCCESS;
MI_LDC_DEV LdcDevId = 0;
MI_LDC_CHN LdcChnId = 0;
MI_LDC_DevAttr_t stDevAttr = {0};
MI_LDC_ChnAttr_t stChnAttr = {};
MI_SYS_ChNPort_t stDstChNPort = {};

stChnAttr.eMode = MI_LDC_WORKMODE_LDC ;
s32Ret = ReadBufFromFile("ldc.json", stChnAttr.pConfigAddr, stChnAttr.u32ConfigSize);
if (s32Ret)
{
```



```

    return s32Ret;
}

stDstChnPort. eModId = E_MI_MODULE_ID_ISP;
stDstChnPort. u32DevId = 0;
stDstChnPort. u32ChnId = 0;
stDstChnPort. u32PortId = 0;

MI_LDC_CreateDevice(LdcDevId, &stDevAttr);
MI_LDC_CreateChannel(LdcDevId, LdcChnId, &stChnAttr);
MI_LDC_AttachToChn(LdcDevId, LdcChnId, &stDstChnPort);
/*退出释放资源*/
ReleaseBuf(stChnAttr. pConfigAddr);
MI_LDC_DetachFromChn(LdcDevId, LdcChnId, &stDstChnPort);
MI_LDC_DestroyChannel(LdcDevId, LdcChnId);
MI_LDC_DestroyDevice(LdcDevId);

```

➤ 相关主题

[MI_LDC_DestroyDevice](#)

2.2. MI_LDC_DestroyDevice

➤ 描述

销毁 LDC device 设备。

➤ 语法

```
MI_S32 MI_LDC_DestroyDevice(MI_LDC_DEV devId);
```

➤ 参数

参数名称	描述	输入/输出
devId	Device id 号	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，参照[错误码](#)。

➤ 需求

- 头文件：mi_ldc.h
- 库文件：libmi_ldc.a

※ 注意

无。

➤ 举例

见 [MI_LDC_CreateDevice](#) 举例。

➤ 相关主题

[MI_LDC_CreateDevice](#).

2.3. MI_LDC_CreateChannel

➤ 描述

创建 LDC channel 通道

➤ 语法

```
MI_S32 MI_LDC_CreateChannel(MI_LDC_DEV devId, MI_LDC_CHN chnId, MI\_LDC\_ChnAttr\_t *pstChnAttr);
```

➤ 参数

参数名称	描述	输入/输出
devId	Device id 号	输入
chnId	Channel 号	输入
pstChnAttr	Channel 属性	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，参照[错误码](#)。

➤ 需求

- 头文件：mi_ldc.h
- 库文件：libmi_ldc.a

※ 注意

- 需在 [MI_LDC_CreateDevice](#) 之后调用。

➤ 举例

见 [MI_LDC_CreateDevice](#) 举例。

➤ 相关主题

[MI_LDC_DestroyChannel](#).

2.4. MI_LDC_DestroyChannel

➤ 描述

销毁 LDC channel 通道

➤ 语法

```
MI_S32 MI_LDC_DestroyChannel(MI_LDC_DEV devId, MI_LDC_CHN chnId);
```

➤ 参数

参数名称	描述	输入/输出
devId	Device id 号	输入
chnId	Ldc Channel 号	输入

- 返回值
- 返回值 $\begin{cases} 0 & \text{成功。} \\ \text{非 } 0 & \text{失败，参照[错误码](#)。} \end{cases}$

- 需求
- 头文件：mi_ldc.h
 - 库文件：libmi_ldc.a

- ※ 注意
- 无。

- 举例
- 见 [MI_LDC_CreateDevice](#) 举例。

- 相关主题
- [MI_LDC_CreateChannel](#)。

2.5. MI_LDC_StartChannel

- 描述
- 启用 LDC channel 通道。

- 语法
- MI_S32 MI_LDC_StartChannel(MI_LDC_DEV devId, MI_LDC_CHN chnId);

- 参数

参数名称	描述	输入/输出
devId	Device id 号	输入
chnId	Ldc Channel 号	输入

- 返回值
- 返回值 $\begin{cases} 0 & \text{成功。} \\ \text{非 } 0 & \text{失败，参照[错误码](#)。} \end{cases}$

- 需求
- 头文件：mi_ldc.h
 - 库文件：libmi_ldc.a

- ※ 注意
- Device 和 Channel 创建之后调用。

- 举例
- 见 [MI_LDC_CreateDevice](#) 举例。

➤ 相关主题

[MI_LDC_StopChannel.](#)

2.6. MI_LDC_StopChannel

➤ 描述

禁用 LDC channel 通道。

➤ 语法

```
MI_S32 MI_LDC_StopChannel(MI_LDC_DEV devId, MI_LDC_CHN chnId);
```

➤ 参数

参数名称	描述	输入/输出
devId	Device id 号	输入
chnId	Ldc Channel 号	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，参照[错误码](#)。

➤ 需求

- 头文件：mi_ldc.h
- 库文件：libmi_ldc.a

※ 注意

无。

➤ 举例

见 [MI_LDC_CreateDevice](#) 举例。

➤ 相关主题

[MI_LDC_StartChannel.](#)

2.7. MI_LDC_GetOutputPortAttr

➤ 描述

获取 LDC output 属性。

➤ 语法

```
MI_S32 MI_LDC_GetOutputPortAttr(MI_LDC_DEV devId, MI_LDC_CHN chnId,  
MI\_LDC\_OutputPortAttr\_t *pstOutputAttr);
```

➤ 参数

参数名称	描述	输入/输出
------	----	-------

devId	Device id 号	输入
chnId	Ldc Channel 号	输入
pstOutputAttr	Output 属性	输出

➤ 返回值

返回值 { 0 成功。
非 0 失败，参照[错误码](#)。

➤ 需求

- 头文件：mi_ldc.h
- 库文件：libmi_ldc.a

※ 注意

创建通道之后可以通过该接口获取输出信息。

➤ 举例

见 [MI_LDC_CreateDevice](#) 举例。

➤ 相关主题

无。

2.8. MI_LDC_SetChnParam

➤ 描述

设置 LDC 通道属性。

➤ 语法

MI_S32 MI_LDC_SetChnParam (MI_LDC_DEV devId, MI_LDC_CHN chnId, [MI_LDC_ChnParam_t](#) *pstChnParam);

➤ 参数

参数名称	描述	输入/输出
devId	Device id 号	输入
chnId	Channel 号	输入
pstChnParam	Channel 配置参数	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，参照[错误码](#)。

➤ 需求

- 头文件：mi_ldc.h
- 库文件：libmi_ldc.a

※ 注意

- 需要在 MI_LDC_CreateChannel 之后使用

➤ 举例

更新 channel 配置时使用，可以用于切换算法模式以及调节镜头参数。

```
MI_LDC_DEV devId = 0;
MI_LDC_CHN chnId = 0;
MI_LDC_ChnParam_t stChnParam = {};
MI_LDC_SensorCalibInfo_t *pCalibInfo = &stChnParam.stCalibInfo;

s32Ret = ReadBufFromFile("ldc.json", &stChnParam.pConfigAddr, &stChnParam.u32ConfigSize);
if (s32Ret)
{
    return s32Ret;
}
s32Ret = ReadBufFromFile("CalibPoly.bin", &pstCalibInfo.pCalibPolyBinAddr, &pstCalibInfo.u32CalibPolyBinSize);
if (s32Ret)
{
    return s32Ret;
}

MI_LDC_SetChnParam(devId, chnId, &stChnParam);

ReleaseBuf(stChnParam.pConfigAddr);
ReleaseBuf(pstCalibInfo.pCalibPolyBinAddr);
```

➤ 相关主题

无。

2.9. MI_LDC_DoLutDirectTask

➤ 描述

执行直接的查表任务，将 buf 直接送给 ldc 硬件处理

➤ 语法

```
MI_S32 MI_LDC_DoLutDirectTask(MI_LDC_DEV devID, MI_LDC_CHN chnId,
MI_LDC_LutTaskAttr_t *pstAttr);
```

➤ 参数

参数名称	描述	输入/输出
devID	设备初始化参数	输入
pstAttr	查表任务配置属性	输入/输出

➤ 返回值

返回值 { 0 成功。
非 0 失败，参照[错误码](#)。

➤ 需求

- 头文件：mi_ldc.h
- 库文件：libmi_ldc.a

※ 注意

无。

➤ 举例

```
MI_S32 s32Ret = MI_SUCCESS;
MI_LDC_DEV devId = 0;
MI_LDC_CHN chnId = 0;
MI_LDC_LutTaskAttr_t stLutTask = {};
MI_LDC_DirectBuf_t stTableX = {}, stTableY = {}, stTableWeight = {};
void *pVirTableX = NULL, *pVirTableY=NULL, *pVirTableWeight = NULL;

#define LUT_TABLE_Init(path, table, ppVir, w, h, __exit_func__)
do { \
    table.ePixelFormat = E_MI_SYS_PIXEL_FRAME_I8; \
    table.u32Width = w; \
    table.u32Height = h; \
    table.u32Stride[0] = w; \

    s32Ret = MI_SYS_MMa_Alloc(0, NULL, w*h, &table.phyAddr[0]); \
    if (s32Ret)
    {
        printf("failed to malloc buf\n");
        goto __exit_func__;
    }
    s32Ret = MI_SYS_Mmap(table.phyAddr[0], w*h, ppVir, false); \
    if (s32Ret)
    {
        printf("failed to Mmap buf\n");
        goto __exit_func__;
    }
    s32Ret = ReadBufFromFile(path, ppVir, w*h);
    if (s32Ret)
    {
        goto __exit_func__;
    }
} while(0)

#define LUT_TABLE_Deinit(table, pVir)
do { \
    if (pVir)
    {
        MI_SYS_Munmap(pVir, table.u32Width * table.u32Height);
    }
    if (table.phyAddr[0])
    {
        MI_SYS_MMA_Free(0, table.phyAddr[0]);
    }
} while(0)

LUT_TABLE_Init("1080p_tableX", stTableX, & pVirTableX, 1920, 1080, __exit__);
LUT_TABLE_Init("1080p_tableY", stTableY, & pVirTableY, 1920, 1080, __exit__);
LUT_TABLE_Init("1080p_tableWeight", stTableWeight, &pVirTableWeight, 1920, 1080, __exit__);

MI_LDC_DoLutDirectTask (devId, chnId, & stLutTask);
```

```
__exit:
    LUT_TABLE_Deinit(stTableX, pVirTableX);
    LUT_TABLE_Deinit(stTableY, pVirTableY);
    LUT_TABLE_Deinit(stTableWeight, pVirTableWeight);
```

- 相关主题
无

2.10. MI_LDC_SetOutputPortAttr

- 描述
配置 LDC output 属性。

- 语法
MI_S32 MI_LDC_SetOutputPortAttr(MI_LDC_DEV devId, MI_LDC_CHN chnId,
[MI_LDC_OutputPortAttr_t](#) *pstOutputAttr);

- 参数

参数名称	描述	输入/输出
devId	Device id 号	输入
chnId	Ldc Channel 号	输入
pstOutputAttr	Output 属性	输入

- 返回值
返回值 $\begin{cases} 0 & \text{成功。} \\ \text{非 } 0 & \text{失败，参照[错误码](#)。} \end{cases}$

- 需求
 - 头文件：mi_ldc.h
 - 库文件：libmi_ldc.a

- ※ 注意
 1. 该接口需要在创建通道之后使用（MI_LDC_CreateChannel）。
 2. 该接口目前仅支持配置 LDC 输出的格式，支持的格式有:NV12, ARGB8888, BGRA8888。
 3. Tiramisu 平台不支持修改格式

- 举例

```
MI_LDC_DEV devId = 0;
MI_LDC_CHN chnId = 0;
MI_LDC_OutputPortAttr_t stLdcPortAttr = {};

MI_LDC_GetOutputPortAttr(devId, chnId, &stLdcPortAttr);
stLdcPortAttr.ePixelFormat = E_MI_SYS_PIXEL_FRAME_ARGB8888;
MI_LDC_SetOutputPortAttr(devId, chnId, &stLdcPortAttr);
```

- 相关主题

无。

2.11. MI_LDC_AttachToChn

➤ 描述

将 LDC 通道绑定到对应的模块通道。

➤ 语法

```
MI_S32 MI_LDC_AttachToChn(MI_LDC_DEV devId, MI_LDC_CHN chnId,
MI_SYS_ChnPort_t * pstChnPort);
```

➤ 参数

参数名称	描述	输入/输出
devId	Device id 号	输入
chnId	Ldc Channel 号	输入
pstChnPort	目标模块通道信息	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，参照[错误码](#)。

➤ 需求

- 头文件：mi_ldc.h
- 库文件：libmi_ldc.a

※ 注意

1. 需要在创建通道之后使用（MI_LDC_CreateChannel）。
2. 仅支持绑定 ISP 模块。

➤ 举例

见 [MI_LDC_CreateDevice](#) 举例

➤ 相关主题

无。

2.12. MI_LDC_DetachFromChn

➤ 描述

将 LDC 通道解绑定。

➤ 语法

```
MI_S32 MI_LDC_DetachFrom(MI_LDC_DEV devId, MI_LDC_CHN chnId,
MI_SYS_ChnPort_t * pstChnPort);
```

➤ 参数

参数名称	描述	输入/输出
------	----	-------

devId	Device id 号	输入
chnId	Ldc Channel 号	输入
pstChnPort	目标模块通道信息	输入

➤ 返回值

返回值 { 0 成功。
非 0 失败，参照[错误码](#)。

➤ 需求

- 头文件：mi_ldc.h
- 库文件：libmi_ldc.a

※ 注意

无。

➤ 举例

见 [MI_LDC_CreateDevice](#) 举例

➤ 相关主题

无。

3. 数据类型

模块相关数据类型定义如下：

数据类型	定义
MI_LDC_DEV	定义 LDC device 的类型
MI_LDC_CHN	定义 LDC channel 的类型
MI_LDC_DevAttr_t	定义 LDC device 属性
MI_LDC_WorkMode_e	定义 LDC 的工作模式
MI_LDC_MapInfoType_e	定义 LDC 映射信息类型
MI_LDC_DisplacementMapInfo_t	定义 LDC 置换图
MI_LDC_SensorCalibInfo_t	定义 LDC sensor 校准参数
MI_LDC_ChnAttr_t	定义 LDC channel 属性
MI_LDC_ChnParam_t	定义 LDC channel 动态参数
MI_LDC_OutputPortAttr_t	定义 LDC output 端口属性
MI_LDC_DirectBuf_t	定义 LDC 输入 buf 类型
MI_LDC_LutTaskAttr_t	定义 LDC 图像查找表任务类型

3.1. MI_LDC_DEV

- 说明
定义 LDC device 的类型
- 定义

```
typedef MI_U32 MI_LDC_DEV;
```
- ※ 注意事项
无。
- 相关数据类型及接口
无。

3.2. MI_LDC_CHN

- 说明
定义 LDC channel 的类型

➤ 定义

```
typedef MI_U32 MI_LDC_CHN;
```

※ 注意事项

无。

➤ 相关数据类型及接口

无。

3.3.

MI_LDC_DevAttr_t

➤ 说明

LDC 设备初始化参数。

➤ 定义

```
typedef struct MI_LDC_DevAttr_s
{
    MI_U32 u32Reserved;
} MI_LDC_DevAttr_t;
```

➤ 成员

成员名称	描述
u32Reserved	保留数据

※ 注意事项

无。

➤ 相关数据类型及接口

[MI_LDC_CreateDevice](#)

3.4.

MI_LDC_WorkMode_e

➤ 说明

LDC 工作模式。

➤ 定义

```
typedef enum
{
    MI_LDC_WORKMODE_LDC = 0x01,
    MI_LDC_WORKMODE_LUT = 0x02,
    MI_LDC_WORKMODE_DIS_GYRO = 0x04,
} MI_LDC_WorkMode_e;
```

➤ 成员

成员名称	描述
MI_LDC_WORKMODE_LDC	LDC 模式
MI_LDC_WORKMODE_LUT	LUT直接查表模式
MI_LDC_WORKMODE_DIS_GYRO	基于陀螺仪的DIS 防抖模式

※ 注意事项
无。

➤ 相关数据类型及接口
[MI_LDC_ChnAttr_t](#)

3.5. MI_LDC_MapInfoType_e

➤ 说明
LDC映射信息类型。

➤ 定义

```
typedef enum
{
    MI_LDC_MAPINFOTYPE_DISPMAP,
    MI_LDC_MAPINFOTYPE_SENSORCALIB,
} MI_LDC_MapInfoType_e ;
```

➤ 成员

成员名称	描述
MI_LDC_MAPINFOTYPE_DISPMAP	置换图
MI_LDC_MAPINFOTYPE_SENSORCALIB	sensor 校准信息

※ 注意事项
无。

➤ 相关数据类型及接口
[MI_LDC_ChnAttr_t](#)

3.6. MI_LDC_DisplacementMapInfo_t

➤ 说明
LDC置换图信息。

➤ 定义

```
typedef struct MI_LDC_DisplacementMapInfo_s
{
    void *pXmapAddr;
    void *pYmapAddr;
    MI_U32 u32XmapSize;
    MI_U32 u32YmapSize;
} MI_LDC_DisplacementMapInfo_t;
```

➤ 成员

成员名称	描述
pXmapAddr	x坐标映射表的起始地址
pYmapAddr	y坐标映射表的起始地址

成员名称	描述
u32XmapSize	x坐标的映射表大小
u32YmapSize	y坐标的映射表大小

※ 注意事项
无。

➤ 相关数据类型及接口

[MI_LDC_ChnAttr_t](#)

[MI_LDC_ChnParam_t](#)

3.7. MI_LDC_SensorCalibInfo_t

➤ 说明

LDC Sensor 校准信息。

➤ 定义

```
typedef struct MI_LDC_SensorCalibInfo_s
{
    void *pCalibPolyBinAddr;
    MI_U32 u32CalibPolyBinSize;
} MI_LDC_SensorCalibInfo_t;
```

➤ 成员

成员名称	描述
pCalibPolyBinAddr	校准信息指针
u32CalibPolyBinSize	校准信息大小

※ 注意事项
无。

➤ 相关数据类型及接口

[MI_LDC_ChnAttr_t](#)

[MI_LDC_ChnParam_t](#)

3.8. MI_LDC_ChnAttr_t

➤ 说明

LDC 设备 Channel 属性。

➤ 定义

```
typedef struct MI_LDC_ChnAttr_s
{
    MI\_LDC\_WorkMode\_e eMode;

    MI_BOOL bUseProjection3x3Matrix;
    MI_S32 as32Projection3x3Matrix[LDC_MAXTRIX_NUM];
```

```

MI_U16 u16UserSliceNum;
MI_U16 u16FocalLength;

void *pConfigAddr;
MI_U32 u32ConfigSize;

MI_LDC_MapInfoType_e eInfoType;
union
{
    MI_LDC_DisplacementMapInfo_t stDispMapInfo;
    MI_LDC_SensorCalibInfo_t stCalibInfo;
};
} MI_LDC_ChnAttr_t;

```

➤ 成员

成员名称	描述
eMode	Channel 的工作模式
bUseProjection3x3Matrix	表明是否使用as32Projection3x3Matrix
as32Projection3x3Matrix	表明陀螺仪摆放的方向（用于DIS模式）
u16UserSliceNum	DIS 垂直方向上切分数目，主要用于Rolling Shutter相机的运动场景，每个Slice会有单独的3x3 Matrix
u16FocalLength	焦距，单位mm（用于DIS模式）
pConfigAddr	LDC 配置信息地址
u32ConfigSize	LDC 配置信息大小
eInfoType	LDC 映射信息类型
stDispMapInfo/stCalibInfo	LDC 置换表/sensor校准信息，包含畸变参数

※ 注意事项
无。

➤ 相关数据类型及接口

[MI_LDC_CreateChannel](#)

3.9. MI_LDC_ChnParam_t

➤ 说明

LDC 设备初始化参数。

➤ 定义

```

typedef struct MI_LDC_ChnParam_s
{
    MI_BOOL bUseProjection3x3Matrix;
    MI_S32 as32Projection3x3Matrix[LDC_MAXTRIX_NUM];
    MI_U16 u16FocalLength;
    void *pConfigAddr;
    MI_U32 u32ConfigSize;
    union

```

```
{
    MI\_LDC\_DisplacementMapInfo\_t stDispMapInfo;
    MI\_LDC\_SensorCalibInfo\_t stCalibInfo;
};
} MI_LDC_ChnParam_t;
```

➤ 成员

成员名称	描述
bUseProjection3x3Matrix	表明是否使用as32Projection3x3Matrix
as32Projection3x3Matrix	表明陀螺仪摆放的方向（用于DIS模式）
u16FocallLength	焦距，单位mm（用于DIS模式）
pConfigAddr	配置信息地址
u32ConfigSize	配置信息大小
stDispMapInfo/ stCalibInfo	LDC 置换表/sensor校准信息，包含畸变参数

※ 注意事项
无。

➤ 相关数据类型及接口

[MI_LDC_SetChnParam](#)

3.10. [MI_LDC_OutputPortAttr_t](#)

➤ 说明

定义 LDC output 端口属性。

➤ 定义

```
typedef struct MI_LDC_OutputPortAttr_s
{
    MI_U16 u16Width;
    MI_U16 u16Height;
    MI_SYS_PixelFormat_e ePixelFormat;
} MI_LDC_OutputPortAttr_t;
```

➤ 成员

成员名称	描述
u16Width	图像的宽度
u16Height	图像的高度
ePixelFormat	图像的像素格式

※ 注意事项
无。

➤ 相关数据类型及接口

[MI_LDC_GetOutputPortAttr](#)

3.11. MI_LDC_DirectBuf_t

➤ 说明

LDC 输入 buf 类型。

➤ 定义

```
typedef struct MI_LDC_DirectBuf_s
{
    MI_SYS_PixelFormat_e ePixelFormat;
    MI_U32 u32Width;
    MI_U32 u32Height;
    MI_U32 u32Stride[2];
    MI_PHY phyAddr[2];
} MI_LDC_DirectBuf_t;
```

➤ 成员

成员名称	描述
ePixelFormat	图像格式
u32Width	图像宽度
u32Height	图像高度
u32Stride	图片每行所占字节数
phyAddr	图像存储物理地址

※ 注意事项

无。

➤ 相关数据类型及接口

[MI_LDC_LutTaskAttr_t](#)

3.12. MI_LDC_LutTaskAttr_t

➤ 说明

LDC 图像查找表任务类型。

➤ 定义

```
typedef struct MI_LDC_LutTaskAttr_s
{
    struct MI_LDC_LutSrcBuf_s
    {
        MI_LDC_DirectBuf_t stTableX;
        MI_LDC_DirectBuf_t stTableY;
        MI_LDC_DirectBuf_t stTableWeight;
    } stSrcBuf;

    MI_LDC_DirectBuf_t stDstBuf;
} MI_LDC_LutTaskAttr_t;
```

➤ 成员

成员名称	描述
stSrcBuf	查找表的数据源以及权重表
stDstBuf	存放查表之后的输出数据

※ 注意事项
无。

➤ 相关数据类型及接口

[MI_LDC_DoLutDirectTask](#)

4. 错误码

表 4-2 LDC API 错误码

错误代码	宏定义	描述
0xA0172001	MI_ERR_LDC_INVALID_DEVID	无效 device id
0xA0172002	MI_ERR_LDC_INVALID_CHNID	无效 channel id
0xA0172003	MI_ERR_LDC_ILLEGAL_PARAM	输入参数非法
0xA0172006	MI_ERR_LDC_NULL_PTR	输入参数空指针错误
0xA0172008	MI_ERR_LDC_NOT_SUPPORT	操作不支持
0xA017200C	MI_ERR_LDC_NOT_NOMEM	内存不足
0xA0172012	MI_ERR_LDC_BUSY	Channel 系统忙
0xA0172015	MI_ERR_LDC_MOD_NOT_INIT	Module 没有初始化
0xA0172016	MI_ERR_LDC_MOD_INITED	Module 已经初始化
0xA0172018	MI_ERR_LDC_PORT_NOT_DISABLE	Port 没有失能
0xA017201F	MI_ERR_LDC_FAIL	操作执行失败
0xA0172300	MI_ERR_LDC_DEV_CREATED	Device 已经创建
0xA0172301	MI_ERR_LDC_DEV_NOT_CREATE	Device 还没有创建
0xA0172302	MI_ERR_LDC_DEV_NOT_DESTROY	Device 没有被销毁
0xA0172303	MI_ERR_LDC_CHN_CREATED	Channel 已经创建
0xA0172304	MI_ERR_LDC_CHN_NOT_CREATE	Channel 还没有创建
0xA0172305	MI_ERR_LDC_CHN_NOT_STOP	Channel 没有被禁用
0xA0172306	MI_ERR_LDC_CHN_NOT_DESTROY	Channel 没有被销毁
0xA0172307	MI_ERR_LDC_PORT_NOT_UNBIND	Port 没有解绑

5. PROCFS 介绍

5.1. cat

➤ 调试信息

```
# cat /proc/mi_modules/mi_ldc/mi_ldc0
```

➤ 调试信息分析

记录当前 LDC 的使用状况以及相关属性、可以动态地获取到这些信息，方便调试和测试。

➤ 参数说明

```
----- start dump LDC DEV info -----
DevID  IRQ_Enable  IsrCnt  IsrNum  TotBlkNum  FramDonCnt  cmdqif
  0      1      553    52      330        1        0
HalInFrmCnt HalDoneFrmCnt HalDoneBlkCnt HalIsrDoneCnt HalIsrErrCnt
      2          2          60          555          0
----- End dump LDC dev 0 info -----
```

参数		描述
Device Info	DevID	设备 ID
	IRQ_Enable	IRQ 使能状况
	IsrCnt	中断发生的次数，在 cmdq mode 下没有参考意义
	IsrNum	中断号
	TotBlkNum	总共需要处理的 ldc 块的数量
	FramDonCnt	处理完帧的数量
	cmdqif	cmdq 接口地址
	HalInFrmCnt	输入到 Hal 的帧总数
	HalDoneFrmCnt	Hal 处理完的帧总数
	HalDoneBlkCnt	Hal 当前帧处理完的块总数
	HalIsrDoneCnt	Hal done 中断发生的次数
	HalIsrErrCnt	Hal Err 中断发生的次数

```
----- start dump LDC CHN info -----
ChnId  workmode  status    InputWH  pixel  stride  InBufCnt  InTodoCnt  atom
  0      ldc      1(1920x1080)  11    1920    2         2         0
EptzInit  OneBinSz    OneBin    TotalBin   LdcDone   EnqTDeq   BlendT
19512us   331071us   595343us  1901949us  466484us  499099us  0us
```

参数		描述
Channel Info	ChnId	Channel ID
	workmode	当前 channel 的工作模式
	status	当前 channel 的状态：0:init 1:start 2:stop 3:destroyed
	InputWH	输入帧的分辨率
	Pixel	输入帧的格式
	Stride	输入帧的 stride[0]
	InBufCnt	前级输入的帧总数量
	IntodoCnt	待处理帧的标号（递增）
	Atom	底层拿住 buffer 数量
	EptzInit	初始化算法所用时间
	OneBinSz	计算最新 view 的 buffer size 所用时间
	OneBin	计算最新 view 所用时间
	TotalBin	LDC 算法计算所有 view 的时间
	LdcDone	LDC 硬件处理时间
	EnqTDeq	拿到前级 buf 到处理完的总时间
	BlendT	alpha blending 所用时间

```
LDC Table Info:
ChnId viewId mode NumRuns ImgFmt OpMode flip
0 0 2P_CM 165 0 3 0
InSize ViewSize OutSize ViewTime
(0000,0000,1920,1080)(0000,0000,1920,0544)(0000,0000,1920,1088)( 219954us)

InBlk OutBlk ScIblk XYScale DeltaXY Crop-En BlkTime
(1568,0536,0096,0114)(0000,0000,0064,0064)(0000,0000,0064,0064)(0001,0001)(0004,0004)(0001)(0001356us)
(1536,0638,0128,0114)(0000,0000,0064,0064)(0064,0000,0064,0064)(0001,0001)(0004,0004)(0001)(0001286us)
(1472,0734,0128,0112)(0000,0000,0064,0064)(0128,0000,0064,0064)(0001,0001)(0004,0004)(0001)(0001230us)
(1376,0822,0160,0104)(0000,0000,0064,0064)(0192,0000,0064,0064)(0001,0001)(0004,0004)(0001)(0001197us)
(1216,0898,0224,0114)(0000,0000,0096,0064)(0256,0000,0096,0064)(0001,0001)(0004,0004)(0001)(0001236us)
(1024,0980,0224,0074)(0000,0000,0096,0064)(0352,0000,0096,0064)(0001,0001)(0004,0004)(0001)(0001680us)
(0800,1012,0256,0044)(0000,0000,0096,0064)(0448,0000,0096,0064)(0001,0001)(0004,0004)(0001)(0001095us)
(0576,0856,0384,0000)(0000,0000,0096,0064)(0544,0000,0096,0064)(0001,0001)(0004,0004)(0001)(0001033us)
(0352,0856,0384,0000)(0000,0000,0096,0064)(0544,0000,0096,0064)(0001,0001)(0004,0004)(0001)(0001033us)
(0128,0856,0384,0000)(0000,0000,0096,0064)(0544,0000,0096,0064)(0001,0001)(0004,0004)(0001)(0001033us)
```

参数		描述
	ChnId	Channel ID
	ViewId	ldc 对应窗口的 Id
	mode	ldc 使用的场景模式
	NumRuns	ldc 当前场景需要处理的总区块数
	ImgFmt	ldc 处理的图像格式
	OpMode	当前 bbox 的模式: 0:2048x16 1:1024x32 2:512x64 3:256x128 4: 128x256
	flip	ldc 的镜像模式
	InSize	当前 view 的输入区域
	ViewSize	当前 view 的输出区域
	OutSize	当前场景的输出区域
	ViewTime	当前 view 的处理时间
	InBlk	当前 blk 的输入区域
	OutBlk	当前 blk 的算法输出区域
	ScIblk	当前 blk 的输出区域
	DeltaXY	当前 blk 对应 delta 表的坐标
	BlkTime	对应 blk 的处理时间

5.2. echo

echo help 查看可用命令

```
# echo help > /proc/mi_modules/mi_ldc/mi_ldc0
```

功能	dump LDC 输出文件
命令	echo writeoutfile [chnid cnt path] > /proc/mi_modules/mi_ldc/mi_ldc0

参数说明	chnid: channel Id
	cnt: dump 的数量
	path: dump 的路径
举例	echo writeoutfile 0 2 /mnt/ > /proc/mi_modules/mi_ldc/mi_ldc0

功能	dump LDC map 文件
命令	echo dumptablebin [chnid view/all path] > /proc/mi_modules/mi_ldc/mi_ldc0
参数说明	chnid: channel id
	view/all: 下载的方式, view: 具体的 viewid, all: 当前场景所有的 view
	path: dump 的路径
举例	echo dumptablebin 0 all /mnt/ > /proc/mi_modules/mi_ldc/mi_ldc0

功能	配置输出属性
命令	echo setout[chnid width height pixel] > /proc/mi_modules/mi_ldc/mi_ldc0
参数说明	chnid: channel id
	width: 输出图像的 width
	height: 输出图像的 height
举例	echo setout 0 1920 1080 11 > /proc/mi_modules/mi_ldc/mi_ldc0

功能	循环播放前帧
命令	echo loopbuf [chnid cnt] > /proc/mi_modules/mi_ldc/mi_ldc0
参数说明	chnid: channel id
	cnt: 循环播放前帧的次数
举例	echo loopbuf 0 11 > /proc/mi_modules/mi_ldc/mi_ldc0

功能	配置 ldc 内部 debug trace
命令	echo debug [chnid str] > /proc/mi_modules/mi_ldc/mi_ldc0
参数说明	chnid: channel id
	str: debug 的方式, off: 关闭, ViewRes, ViewLock, ChnRdLock, ChnWrLock, DisGyro, Eptz
举例	echo debug 0 ChnRdLock > /proc/mi_modules/mi_ldc/mi_ldc0

功能	配置单个 ldc map
命令	echo setbin [chnid path] > /proc/mi_modules/mi_ldc/mi_ldc0
参数说明	chnid: channel id
	path: map 的路径

举例	<code>echo setbin /mnt/ldc.bin > /proc/mi_modules/mi_ldc/mi_ldc0</code>
----	--